SOFTWARE ENGINEERING

LAB 7 REPORT

Fuzz Testing for Input Validation Bugs

Name	Sanapala Venkata Maneesh
SRN	PES1UG24AM247
Programming Language	Python 3.11.9
Testing Tools	Hypothesis 6.112.1 and Pytest 9.2.0

This report documents the execution of Lab 7, in which Hypothesis was used to generate unexpected inputs for three functions in a bookstore trade-in processor. The initial fuzz tests exposed input-handling failures. The application was then updated and the same fuzz tests were executed again.

Submitted as part of the Software Engineering laboratory assignment.

1. Objective

The objective of this lab is to explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in a Python script using the Hypothesis fuzzing library. Fuzz testing supplies random, unexpected or malformed inputs to discover crashes and improve robustness.

2. Tools and Environment

Component	Version / Details
Operating system	Windows (PowerShell terminal)
Python	3.11.9
Hypothesis	6.112.1
Pytest	9.2.0
Project files	processor.py and test_processor.py

3. Functions Tested

verify_isbn10_format(isbn_str) – Processes an ISBN-10 string and returns its group code, first digit and checksum character.

calculate_depreciation_factor(book_age_str) – Calculates a depreciation factor from an age/lifespan ratio such as 3/10.

lookup_condition_multiplier(condition_code) – Returns the multiplier associated with MINT, LIKE_NEW, VERY_GOOD, GOOD or POOR.

4. Fuzz Test Design

The starter test file was completed using Hypothesis' @given decorator and the strategy st.text() combined with st.none(). Each test calls one target function inside a try/except block. If the function raises an exception, the test fails and reports the generated input and exception. This checks whether the functions crash when given generated strings or None.

```
@given(st.one_of(st.text(), st.none()))
def test_verify_isbn10_format_no_crash(s):
    try:
        verify_isbn10_format(s)
    except Exception as e:
        assert False, f"ISBN function crashed with input {s!r}: {e}"

# The same pattern is used for the depreciation and
# condition-multiplier functions.
```

Hypothesis generated 100 examples per test in the reported run. It also minimised failing inputs to small examples, making the failures easier to analyse.

5. Initial Fuzz Testing Results

The first meaningful fuzz-test run was executed against the original, unmodified processor.py. All three tests failed. The output showed distinct failure examples and exception types for each function.

Function	Examples / exceptions observed
ISBN format	':' → ValueError; '0' and '' → IndexError; None → AttributeError
Depreciation	'/' → ValueError; '' → ValueError; None → AttributeError
Condition lookup	'' → KeyError; None → AttributeError

Initial Pytest result

```
collected 3 items

test_verify_isbn10_format_no_crash       FAILED
test_calculate_depreciation_factor_safe  FAILED
test_lookup_condition_multiplier_safe    FAILED

3 failed in 5.23s
```

Interpretation: The ISBN function assumed a usable string of the expected length and valid characters. The depreciation function assumed a correctly formatted ratio with valid numeric values and a non-zero lifespan. The condition lookup assumed a non-empty, recognised condition string.

6. Corrections Applied

Function	Defensive changes
verify_isbn10_format()	Checks that input is a string, removes hyphens and whitespace, requires 10 characters, checks the first nine
calculate_depreciation_factor()	Checks the input type and ratio structure; validates numeric, finite values; rejects negative age and non-posi
lookup_condition_multiplier()	Checks the input type and uses dictionary get() so unknown or empty condition codes return None instead o

For invalid inputs, the corrected functions return None. The existing valid input behaviour and condition multiplier values were preserved.

7. Final Execution and Verification

After updating processor.py, the application was run again with valid sample inputs. The reported output remained correct:

```
ISBN input:       0-306-40615-2
ISBN result:      Group 306: Starts 0, Checksum 2

Age/lifespan:     3/10
Depreciation:     0.7

Condition:        MINT
Multiplier:       1.25
```

Final Hypothesis / Pytest result

```
collected 3 items

test_verify_isbn10_format_no_crash       PASSED
test_calculate_depreciation_factor_safe  PASSED
test_lookup_condition_multiplier_safe    PASSED

3 passed in 0.45s
```

Additional valid-input checks reported the expected values for all five supported condition codes: MINT = 1.25, LIKE_NEW = 1.0, VERY_GOOD = 0.8, GOOD = 0.6 and POOR = 0.3. Manual invalid-input checks for None, a zero lifespan ratio and an unknown condition returned None without an exception.

8. Conclusion

This lab demonstrated how property-based fuzz testing can reveal input validation defects that may not be covered by ordinary example-based tests. Hypothesis generated unexpected strings and None values and exposed failures in all three application functions. Defensive checks and safe lookup and calculation logic were then introduced. On re-running the same three fuzz tests, all tests passed. The exercise improved the robustness of the bookstore trade-in processor and demonstrated the value of automated edge-case testing.

Note on evidence: This report records the terminal outputs and test results shared during the lab execution. It contains formatted execution excerpts, not pasted terminal screenshot images.

9. Files Submitted

processor.py – corrected application; test_processor.py – Hypothesis fuzz tests; requirements.txt – Python dependencies; README.md – execution instructions; screenshots – execution evidence to be added separately if required by the instructor.